

Progetto di Laboratorio

AREA HADAMARD GATE

Sezione 2 del Report: **Risultato del Linter**

Strumento: **VHDL Style Guide (VSG) 3.35.0**

2. Sezione 2 — Risultato del Linter

2.1 Cos'è un linter e perché si usa

Un **linter** è uno strumento di *analisi statica* del codice: esamina il sorgente senza eseguirlo e segnala violazioni di regole stilistiche, sintattiche o di best practice. Per il VHDL questo è particolarmente importante perché:

- elimina ambiguità che il sintetizzatore potrebbe interpretare in modo diverso dal simulatore;
- impone uno stile uniforme (indentazione, nomi, spaziatura, posizionamento delle parentesi) che migliora la **leggibilità** e la **manutenibilità**;
- previene errori comuni (mancanza di *label* sui process, *if/then* mal formati, default value su signal sintetizzabili, etc.).

Strumento scelto

È stato utilizzato **VSG — VHDL Style Guide**, un linter open-source mantenuto su GitHub e adottato in numerosi progetti industriali (anche dal flusso CI/CD di OpenHW Group / CORE-V). VSG implementa centinaia di regole conformi alle convenzioni IEEE 1076 e supporta sia il *check* sia il *fix* automatico. La versione utilizzata è la **3.35.0**.

Configurazione utilizzata

Per il presente progetto è stato adottato il ruleset di default, con la sola eccezione della regola *signal_007* (vieta il valore di default su un signal), disabilitata **solo per il testbench** in quanto, in codice non sintetizzabile, l'inizializzazione di un signal è una pratica corretta e diffusa. Il file di configurazione *vsg_config.yaml* è il seguente:

```
rule:
  signal_007:
    disable: true
```

2.2 Comando eseguito

Il linter è stato lanciato dalla cartella radice del progetto con il seguente comando (PowerShell):

```
vsg -c vsg_config.yaml `
  -f src/hadamard_core.vhd `
    src/hadamard_top.vhd `
      sim/tb_hadamard.vhd
```

L'opzione **--fix** è stata utilizzata in un passaggio intermedio per applicare automaticamente i fix di formattazione (spaziatura, indentazione, layout di if/then, posizionamento parentesi). Il comando di sola verifica usato per produrre i risultati finali NON modifica i file.

2.3 Risultati: prima e dopo l'auto-fix

Prima dell'applicazione dei fix automatici, il linter ha individuato **44 violazioni** di tipo esclusivamente **stilistico** (formattazione e leggibilità), nessuna delle quali di natura logica o semantica.

Sintesi PRIMA del fix

File	Errors	Warnings
src/hadamard_core.vhd	15	0
src/hadamard_top.vhd	6	0
sim/tb_hadamard.vhd	23	0
TOTALE	44	0

Categorie di violazione rilevate

Regola	# Occ.	Descrizione breve
whitespace_011	15	Spaziatura attorno a +, -, *
port_007/008	6	Allineamento delle keyword in / out
if_002	4	Condizione non racchiusa fra ()
component_022	2	Mancanza di component_simple_name
process_012	3	Mancanza della keyword 'is' dopo process
process_018	1	Process senza label sul 'end process'
instantiation_033	2	Mancanza della keyword 'component'
function_019/020	3	Layout parentesi/designator funzione
procedure_013/014	3	Layout parentesi/designator procedura
generic_map_*	2	Formattazione generic map
port_map_*	5	Formattazione port map
if_020 / if_024	2	Layout di if / then / end if
report_statement_002	1	severity su riga separata
signal_007	1	Default su signal (solo testbench)

Sintesi DOPO il fix

✓ Linter pulito: 0 errors, 0 warnings su tutti e tre i file (888 regole verificate per ciascun file).

File	Errors	Warnings	Rules checked
src/hadamard_core.vhd	0	0	888
src/hadamard_top.vhd	0	0	888
sim/tb_hadamard.vhd	0	0	888
TOTALE	0	0	—

2.4 Estratto dell'output del linter (finale, dopo --fix)

Di seguito è riportato l'output testuale prodotto da VSG dopo l'applicazione dei fix automatici. Per ogni file è indicato il numero di regole verificate e il numero di violazioni residue:

```
=====
File:  src/hadamard_core.vhd
=====
Phase 7 of 7... Reporting
Total Rules Checked: 888
Total Violations:    0
  Error   :    0
  Warning :    0

=====
File:  src/hadamard_top.vhd
=====
Phase 7 of 7... Reporting
Total Rules Checked: 888
Total Violations:    0
  Error   :    0
  Warning :    0

=====
File:  sim/tb_hadamard.vhd
=====
Phase 7 of 7... Reporting
Total Rules Checked: 888
Total Violations:    0
  Error   :    0
  Warning :    0
```

2.5 Discussione dei risultati

L'analisi statica del codice VHDL ha confermato la **qualità del progetto** sotto il profilo dello stile:

- 1. Nessun errore semantico.** Tutte le violazioni iniziali (44 in totale) sono di natura puramente stilistica (spaziatura, indentazione, layout). Nessuna di esse avrebbe potuto generare un comportamento errato in simulazione o in sintesi.
- 2. Cleanup automatico.** Il fix automatico di VSG ha riformattato il codice in conformità allo standard, senza intervento manuale e senza alterare la logica. Dopo il fix, le 888 regole vengono rispettate al 100%.
- 3. Riproducibilità.** Il file di configurazione *vsg_config.yaml* è versionato nel repository del progetto: chiunque clonerà il codice potrà rieseguire il linter ottenendo lo stesso risultato.
- 4. Coerenza con la sintesi.** Il check di VSG copre molte delle stesse verifiche del *Report Methodology* di Vivado (CDC, completeness di sensitivity list, default values, formattazione process e generate). Avere un linter pulito riduce in modo significativo il numero di warning attesi nella fase di sintesi su Vivado.

Conclusione

Il design è LINT-CLEAN: zero violazioni stilistiche o sintattiche residue. Si può quindi procedere con la sintesi e l'implementazione su Vivado in condizioni di pulizia ottimale.